



郑州大学
ZHENGZHOU UNIVERSITY

计算复杂性理论

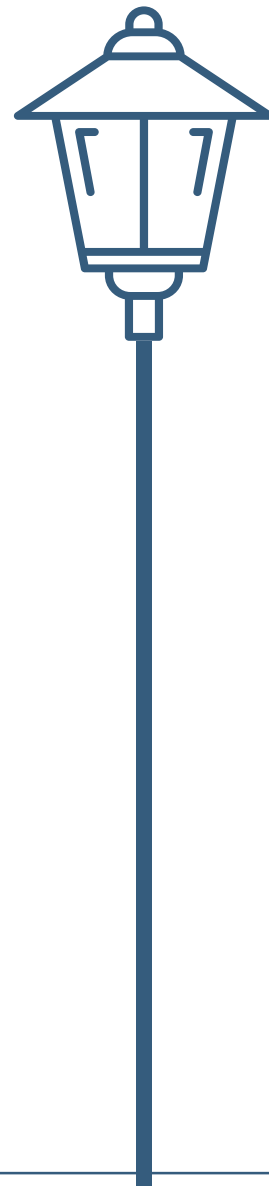
PART 01 基本问题

- 1、简介
- 2、计算模型
- 3、可计算理论
- 4、计算分析
- 5、复杂性类

1、简介

计算复杂性理论 (Computational complexity theory) 是计算机科学中研究各类问题计算复杂性的分支学科，它使用数学方法对计算中所需的各种资源的耗费作定量的分析，并研究各类问题之间在计算复杂程度上的相互关系和基本性质，是算法分析的理论基础。

所研究的资源中最常见的是**时间**（要通过多少步才能解决问题）和**空间**（在解决问题时需要多少内存）。



2 计算模型：

一个算法的实现总要借助于实际的计算模型。

- 图灵机 (Turing machine) :

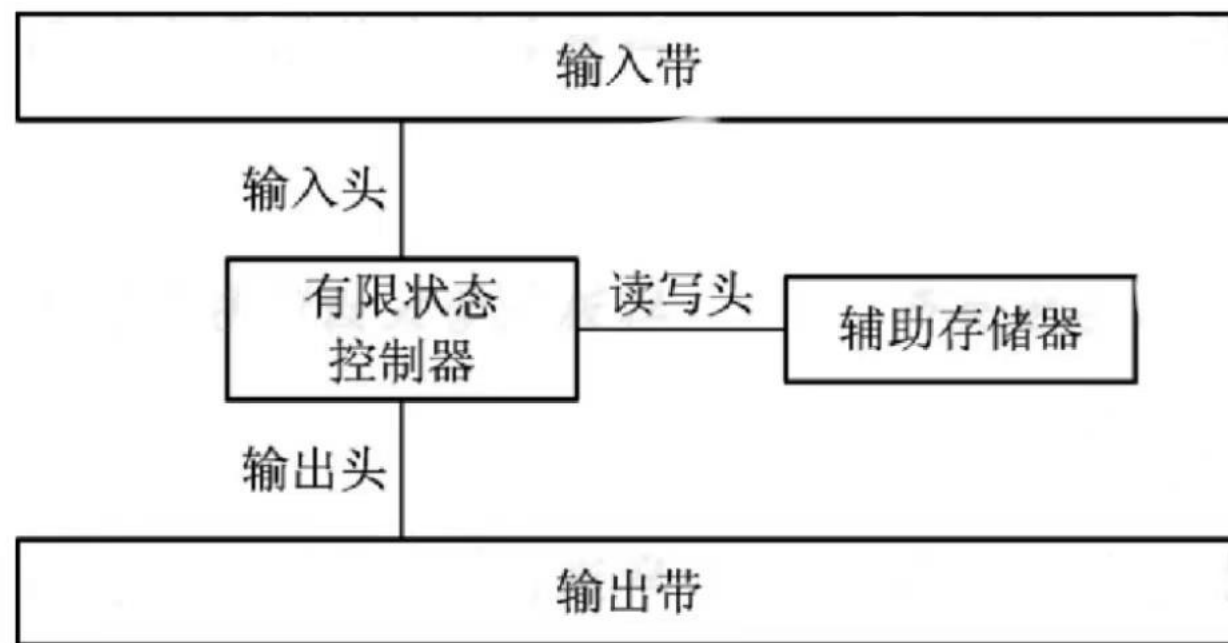
有著名的Chuch 论题：“可计算就是图灵机可计算”

确定型和非确定型

计算资源：时间、空间

- 电路 (circuit) :

计算资源：电路大小

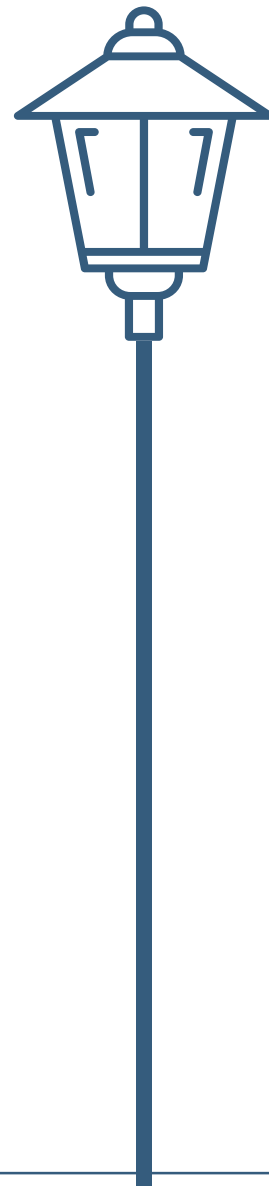


可计算理论：

通过建立计算的数学模型，区分可计算与不可计算。

可计算函数：能够在抽象计算机上编出程序计算其值的函数，这样的程序成为算法

可计算性：指在一个实际问题是否可以使用计算机来解决（能解决一定是有限步内解决）

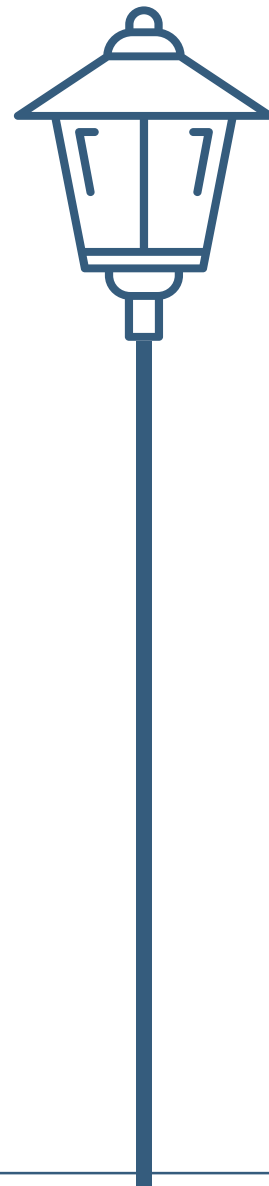


可计算理论和计算复杂性理论区别：

可计算理论：关注的是从原则上可以用算法求解的问题，不管需要多少资源
(把问题区分为可计算和不可计算)

计算复杂性理论：专注于理解为什么对于某类问题不存在有效的算法

算法理论：专注于设计有效的算法



算法分析

时间复杂性：是指求解一个问题所需多少时间，是衡量一个算法优劣的重要参数。时间复杂度越小，说明该算法效率越高，则该算法越有价值。

空间复杂性：是指求解一个问题所需多少空间，它是算法优劣的重要度量指标。空间复杂度越小，算法越好。

渐进时间复杂度(时间复杂度):

$$T(n) = O(f(n))$$

n : 问题规模

$T(n)$: 语句总的执行次数

$f(n)$: 问题规模 n 的某个函数

例:

$$O(2n^2 + n + 1) = O(3n^2 + n + 3) = O(7n^2 + n) = O(n^2)$$

- 多项式时间算法：指时间复杂性函数为 $O(p(n))$ 的算法，其中 $p(n)$ 为 n 的多项式
- 指数时间算法：指时间复杂性为 $O(a^{h(n)})$ 、 $O(n^{\log n})$ 或 $O(e^{\sqrt{n \ln n}})$

表 1－1 不同类型算法的运行时间(假设计算机执行一条指令的时间为 1 μs)

算法类别		复杂性	$n=10^6$ 时的运算次数	实际运行时间
多项式时间算法	常数	$O(1)$	1	1 μs
	线性	$O(n)$	10^6	1 s
	二次	$O(n^2)$	10^{12}	11.6 天
	三次	$O(n^3)$	10^{18}	32 000 年
指数时间算法		$O(2^n)$	$10^{301\ 030}$	约 $3 \times 10^{301\ 016}$ 年

多项式时间算法:

常数阶 $O(1)$

线性阶 $O(n)$

对数阶 $O(\log N)$

线性对数阶 $O(n \log N)$

平方阶 $O(n^2)$

立方阶 $O(n^3)$

K次方阶 $O(n^k)$

指数阶 (2^n)

- 常数级: $O(1)$

```
int sum = 0, n = 100;  
sum = (1 + n) * n / 2;  
printf("%d",sum);
```
- 线性阶: $O(n)$

```
int i;  
for(i = 0; i < n; i++){
```
- 对数阶: $O(\log N)$

```
int count = 1;  
while (count < n){  
count = count * 2; }
```

多项式时间算法:

常数阶 $O(1)$

线性阶 $O(n)$

对数阶 $O(\log N)$

线性对数阶 $O(n \log N)$

平方阶 $O(n^2)$

立方阶 $O(n^3)$

K次方阶 $O(n^k)$

指数阶 2^n

平方阶: $O(n^2)$

```
int i, j;
for(i = 0; i < n; i++){
    for(j = 0; j < n; j++){
        /*时间复杂度为O(1)的程序步骤序列*/
    }
}
```

立方阶 $O(n^3)$

```
for(i=0;i<n;i++)
{
    for(j=0;j<i;j++)
    {
        for(k=0;k<j;k++)
            x=x+2;
    }
}
```

一些计算的规则

1、加法规则

$$T(n,m) = T1(n) + T2(m) = O(\max\{f(n), g(m)\})$$

2、乘法规则

$$T(n,m) = T1(n) * T2(m) =$$

3、比较

$$c(\text{常数}) < \log n < n < n \cdot \log n$$

各种常用排序算法						
类别	排序方法	时间复杂度			空间复杂度	稳定性
		平均情况	最好情况	最坏情况	辅助存储	
插入排序	直接插入	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	shell排序	$O(n^{1.3})$	$O(n)$	$O(n^2)$	$O(1)$	不稳定
选择排序	直接选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
	堆排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	不稳定
交换排序	冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	稳定
	快速排序	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n^2)$	$O(n \log_2 n)$	不稳定
归并排序		$O(n \log_2 n)$	$O(n \log_2 n)$	$O(n \log_2 n)$	$O(1)$	稳定
基数排序		$O(d(r+n))$	$O(d(n+rd))$	$O(d(r+n))$	$O(rd+n)$	稳定

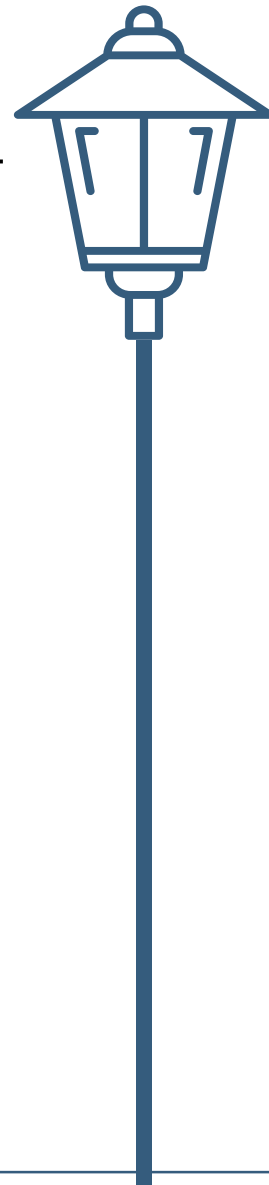
注：基数排序的复杂度中，r代表关键字的基数，d代表长度，n代表关键字的个数

复杂性类

将计算问题按照在不同计算模型下所需资源的不同予以分类，从而得到一个对算法问题“**难度**”的类别，就是复杂性理论中复杂性类概念的来源。

P：在确定性图灵机上所需时间不会超过一个确定的多项式

NP：“确定性图灵机”改为“不确定性图灵机”



CONTENTS

01/

P问题

02/

NP问题

03/

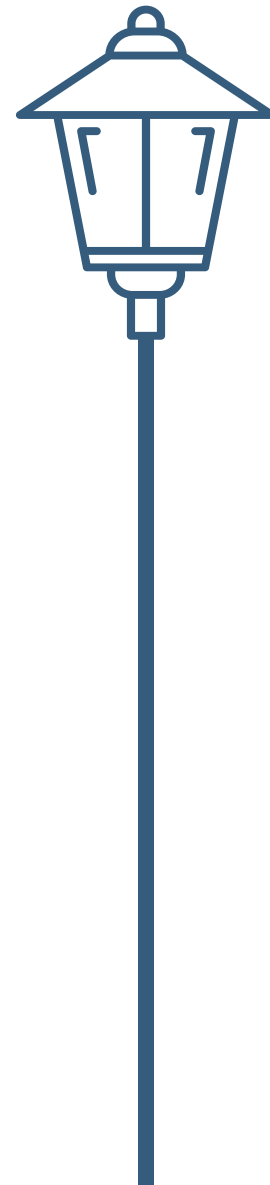
NP完全问题 (NPC问题)

04/

P、NP、NPC之间的关系

多项式时间复杂度

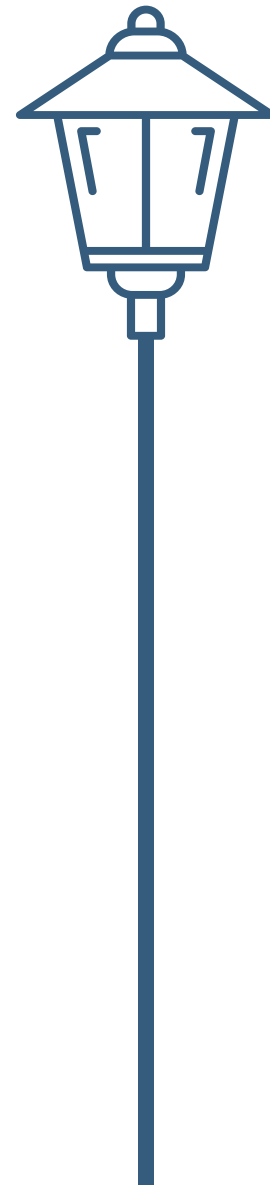
- 定义： 解决问题需要的时间与问题的规模之间是多项式关系。
- 例如： $O(n \log(n))$ 、 $O(n^3)$ 都是多项式时间
 $O(a^n)$ 和 $O(n!)$ 是非多项式级的



P问题

《算法导论》给出的定义：在多项式时间内可解的问题为 P 问题（Polynomial Problem，多项式问题）。

例如：快速排序、堆排序、冒泡排序和直接选择排序算法

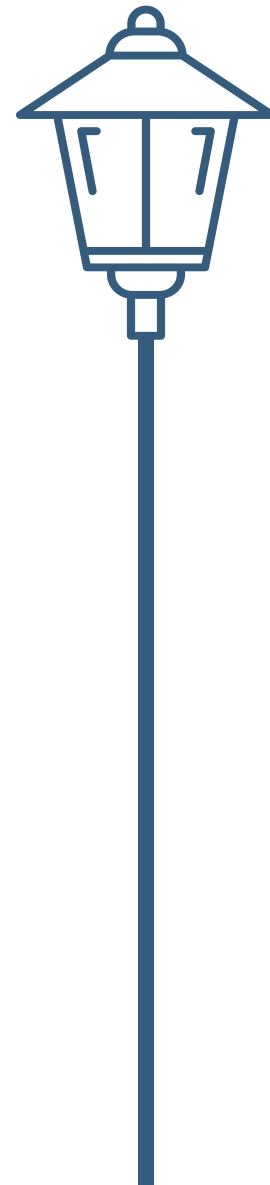


NP问题

NP问题（非确定性多项式问题），指问题只能通过验证给定的猜测是否正确来求解。

多项式指的是验证猜测可在多项式时间内完成，非确定性指的是问题只能通过验证猜测来解，而不能直接求解。

例如：Hamilton回路、大合数的质因数分解



NP完全性问题

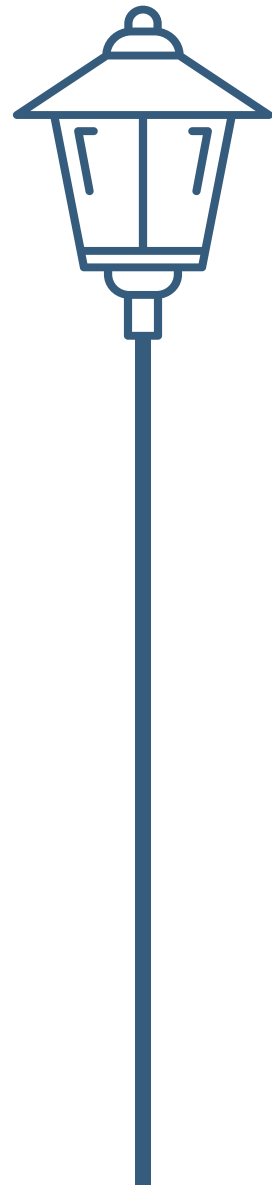


NP=P ?

约化

约化的概念：

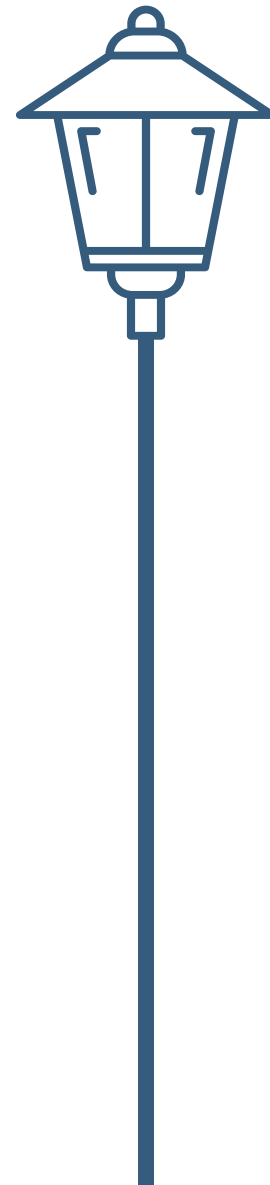
如果能找到这样一个变化法则，对任意一个程序 A 的输入，都能按这个法则变换成程序 B 的输入，使两程序的输出相同，那么我们说，问题 A 可约化为问题 B ，即可以用问题 B 的解法解决问题 A ，或者说，问题 A 可以“变成”问题 B 。



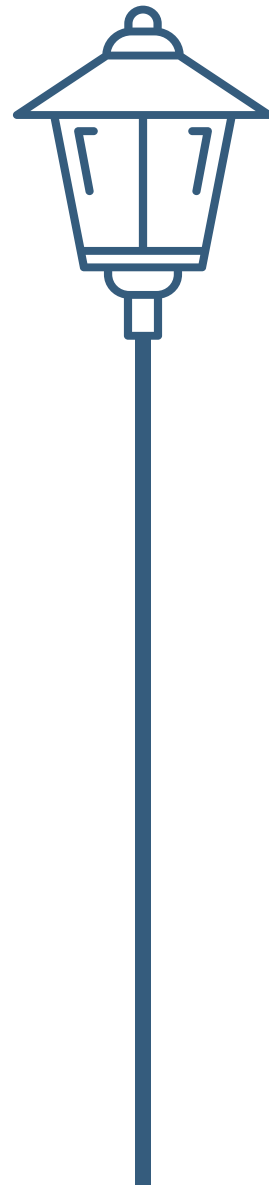
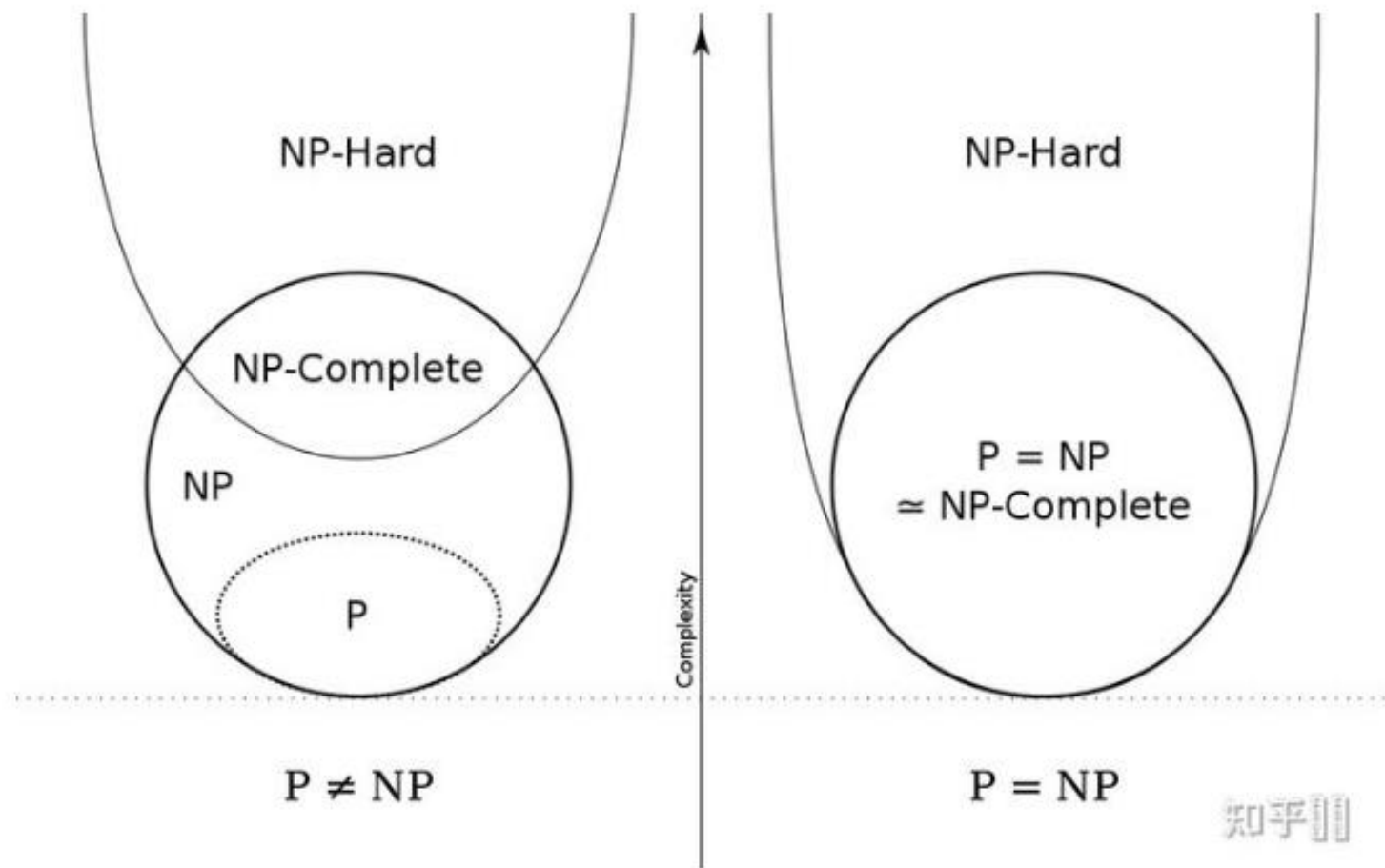
约化

A: 一元一次方程 B: 一元二次方程

- 约化具有传递性
- “问题A可约化为问题B”有一个重要的直观意义：B的时间复杂度高于或者等于A的时间复杂度。

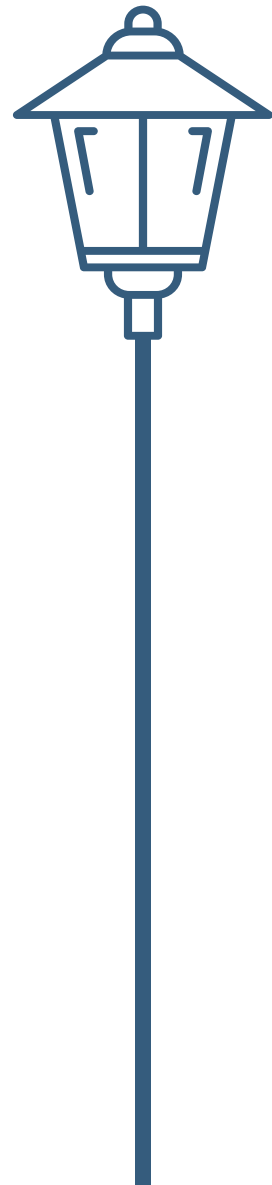


NPC问题

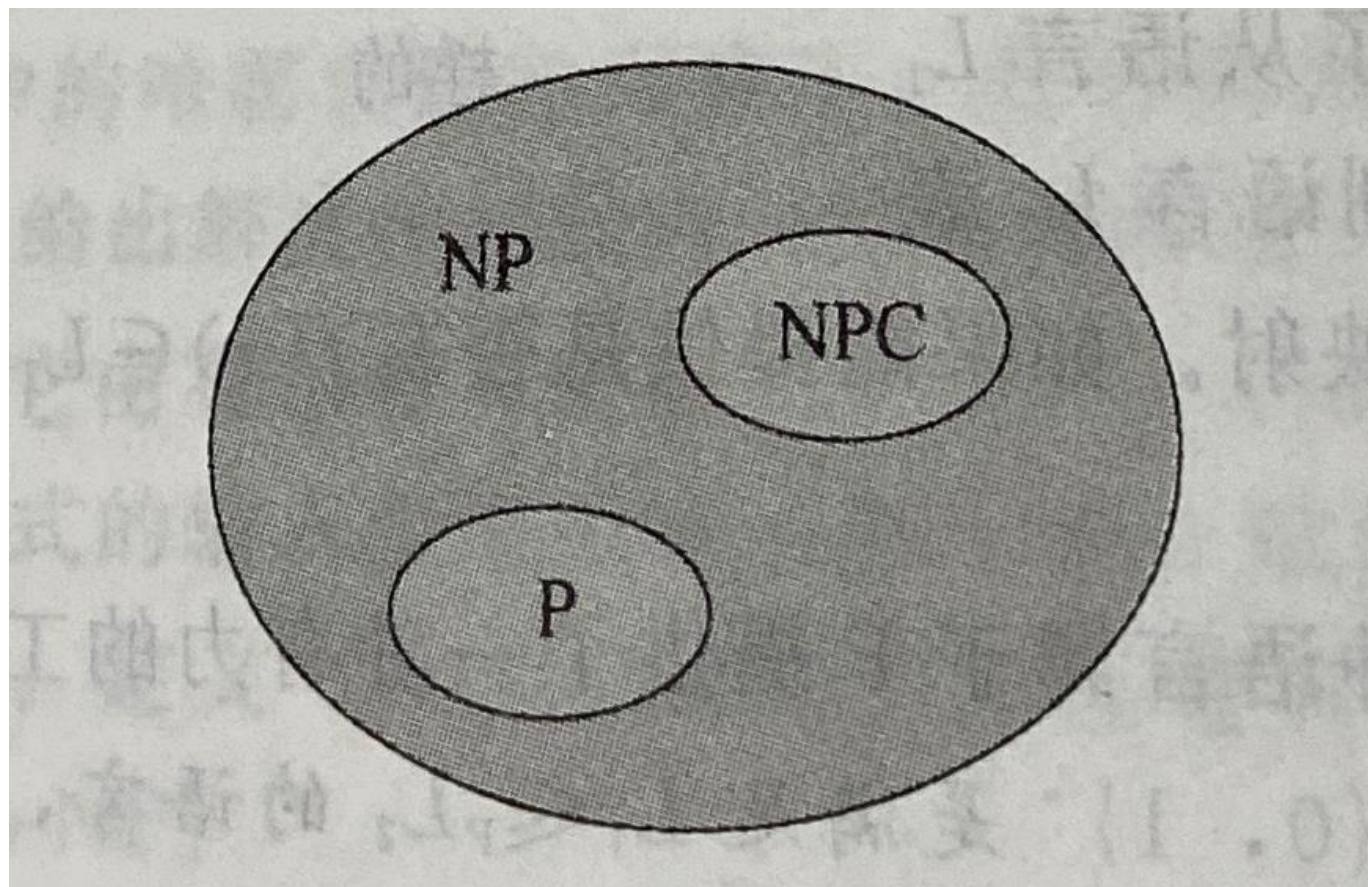


NPC问题

- NPC 问题是指满足下面两个条件的问题：
 - (1) 它是一个 NP 问题；
 - (2) 所有的 NP 问题都可以用多项式时间约化到它。
- 例如：哈密顿回路问题、旅行商问题



P、NP、NPC之间的关系





Thanks!